



CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS

**Implementação e Teste de Aplicação Cliente–Servidor com Criptografia Utilizando
Sockets UDP/TCP e Regras de Firewall IPTables**

Gabriela Honório de Freitas

1. Introdução

Este relatório descreve a implementação de uma aplicação distribuída composta por dois processos:

- Cliente: envia uma mensagem ao servidor.
- Servidor: recebe a mensagem, aplica uma criptografia simples (ex.: cifra de César) e devolve ao cliente.

A aplicação foi implementada em duas versões:

1. Comunicação via UDP (datagram sockets)
2. Comunicação via TCP (stream sockets)

Também foram configuradas regras de firewall utilizando IPTables para controlar o acesso do cliente ao servidor, alternando permissões entre UDP e TCP.

2. Estrutura da Aplicação

2.1. Criptografia Utilizada

Para fins didáticos, foi utilizada uma cifra simples: Cifra de César (deslocamento de 3 caracteres).

3.1. Servidor UDP

```
import java.net.*;

public class ServidorUDP {
    public static void main(String[] args) throws Exception {

        DatagramSocket socket = new DatagramSocket(6789);
        System.out.println("Servidor UDP esperando mensagens...");

        byte[] bufferReceber = new byte[1024];
        byte[] bufferEnviar;

        while (true) {
            DatagramPacket pacoteReceber = new DatagramPacket(bufferReceber,
bufferReceber.length);
            socket.receive(pacoteReceber);
```

```

        String mensagem = new String(pacoteReceber.getData(), 0,
pacoteReceber.getLength());
        System.out.println("Mensagem recebida: " + mensagem);

        // CRIPTOGRAFIA (CÉSAR +3)
        String criptografada = criptografar(mensagem);

        bufferEnviar = criptografada.getBytes();

        InetAddress ipCliente = pacoteReceber.getAddress();
        int portaCliente = pacoteReceber.getPort();

        DatagramPacket pacoteEnviar = new DatagramPacket(bufferEnviar,
bufferEnviar.length, ipCliente, portaCliente);
        socket.send(pacoteEnviar);
    }
}

    public static String criptografar(String msg) {
        StringBuilder sb = new StringBuilder();
        for (char c : msg.toCharArray()) sb.append((char) (c + 3));
        return sb.toString();
    }
}

```

3.2. Cliente UDP

```

import java.net.*;

public class ClienteUDP {
    public static void main(String[] args) throws Exception {

        InetAddress serv = InetAddress.getByName("IP_DO_SERVIDOR");
        DatagramSocket s = new DatagramSocket();

        String msg = "OLA SERVIDOR UDP";
        byte[] m = msg.getBytes();

        DatagramPacket req = new DatagramPacket(
            m, m.length, serv, 6789
        );

        s.send(req);

        byte[] buffer = new byte[1000];
        DatagramPacket resp = new DatagramPacket(buffer, buffer.length);
        s.receive(resp);
    }
}

```

```

        System.out.println("* Resposta criptografada: " +
            new String(resp.getData(), 0, resp.getLength()));

        s.close();
    }
}

```

3.4. Servidor TCP

```

import java.io.*;
import java.net.*;

public class ServidorTCP {

    public static String cifra(String s) {
        StringBuilder sb = new StringBuilder();
        for(char c : s.toCharArray()) sb.append((char)(c+3));
        return sb.toString();
    }

    public static void main(String[] args) throws Exception {
        ServerSocket escuta = new ServerSocket(6790);
        System.out.println("*** Servidor TCP na porta 6790");

        while(true) {
            Socket cliente = escuta.accept();
            System.out.println("*** Conexão aceita de: " + cliente.getInetAddress());

            BufferedReader ent = new BufferedReader(new
InputStreamReader(cliente.getInputStream()));
            PrintWriter sai = new PrintWriter(cliente.getOutputStream(), true);

            String msg = ent.readLine();
            System.out.println("*** Recebido: " + msg);

            sai.println(cifra(msg));
            cliente.close();
        }
    }
}

```

3.5. Cliente TCP

```

import java.io.*;
import java.net.*;

public class ClienteTCP {

```

```
public static void main(String[] args) throws Exception {  
  
    Socket s = new Socket("IP_DO_SERVIDOR", 6790);  
  
    PrintWriter sai = new PrintWriter(s.getOutputStream(), true);  
    BufferedReader ent = new BufferedReader(new  
InputStreamReader(s.getInputStream()));  
  
    sai.println("OLA SERVIDOR TCP");  
    String resposta = ent.readLine();  
  
    System.out.println("*** Resposta criptografada: " + resposta);  
  
    s.close();  
}  
}
```

10. Testes na Máquina Real com Comunicação TCP e UDP (Foco na Criptografia)

Após desenvolver e validar a aplicação em ambiente de teste, os programas Cliente e Servidor — nas versões TCP e UDP — foram executados em uma máquina real para confirmar o funcionamento da comunicação e da criptografia aplicada.

10.1. Teste da Comunicação TCP (Java)

Compilação dos programas

```
javac ClienteTCP.java
```

```
javac ServidorTCP.java
```

Execução do servidor

```
java ServidorTCP
```

Execução do cliente

```
java ClienteTCP
```

Resultados observados

- O cliente estabeleceu conexão com o servidor com sucesso.
- A mensagem digitada foi enviada corretamente ao servidor.
- O servidor aplicou a criptografia (Cifra de César, deslocamento +3).
- A resposta criptografada retornou ao cliente e foi exibida.
- O teste confirmou o funcionamento correto da comunicação TCP e da criptografia.

```

cefet@LAB-6201B-14:~$ javac ClienteTCP.java
cefet@LAB-6201B-14:~$ javac ClienteTCP.java
cefet@LAB-6201B-14:~$ java ClienteTCP
Erro: Conexão recusada
cefet@LAB-6201B-14:~$ java ClienteTCP
Erro: Conexão recusada
cefet@LAB-6201B-14:~$ javac ClienteTCP.java
cefet@LAB-6201B-14:~$ java ClienteTCP
Digite uma mensagem para enviar ao servidor: gabriela
Resposta do servidor: jdeulhod
cefet@LAB-6201B-14:~$

```

10.2. Teste da Comunicação UDP (Java)

Compilação dos programas

```
javac ClienteUDP.java
```

```
javac ServidorUDP.java
```

Execução do servidor

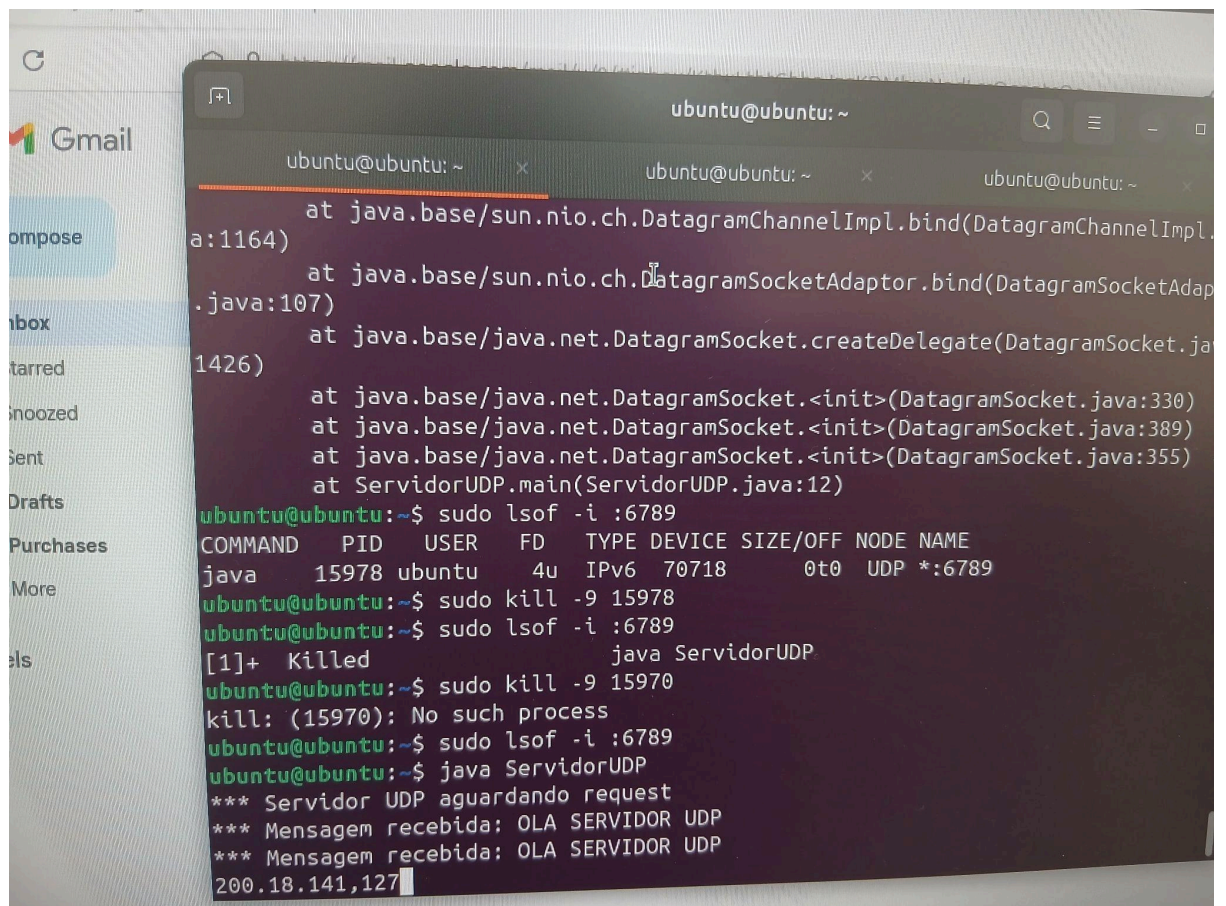
```
java ServidorUDP
```

Execução do cliente

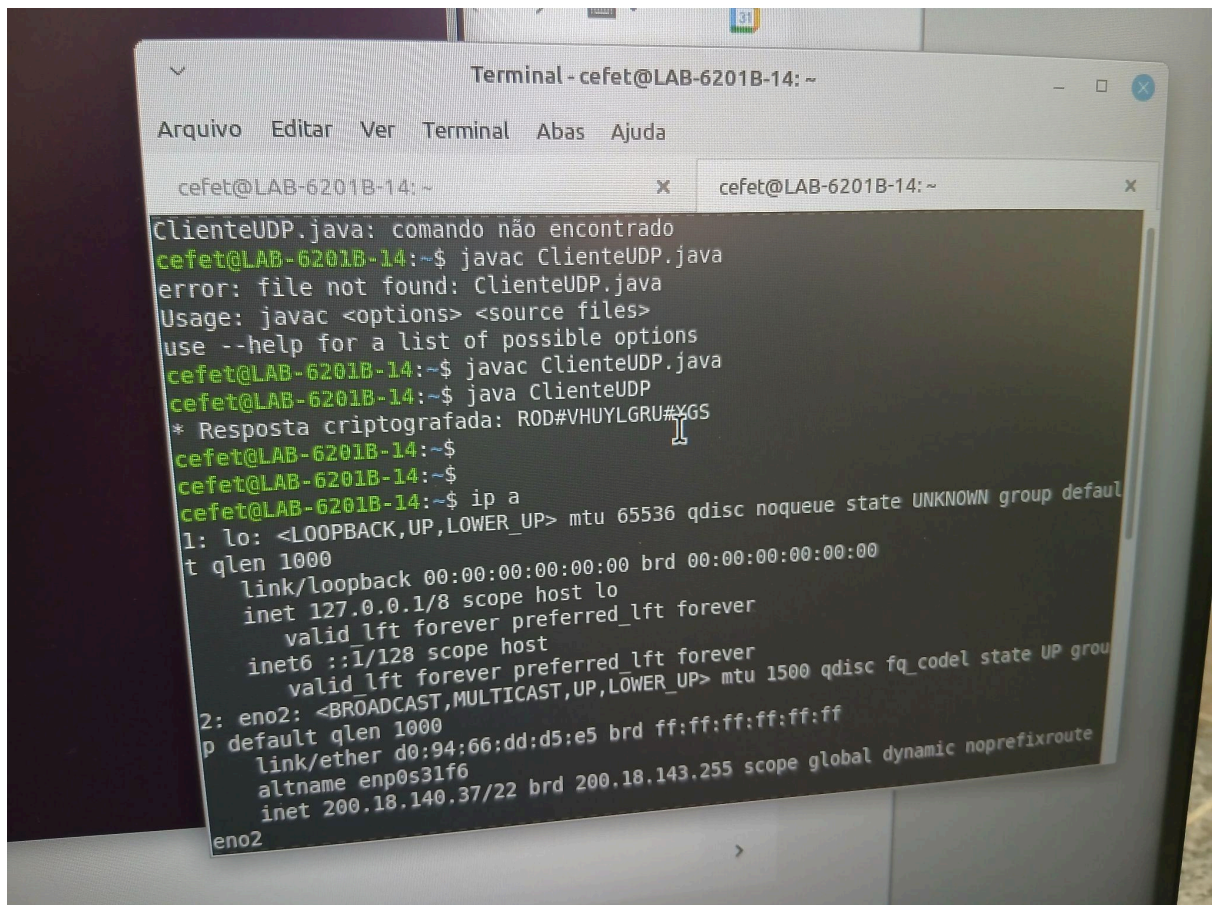
java ClienteUDP

Resultados observados

- O cliente enviou a mensagem ao servidor utilizando UDP.
- O servidor recebeu o datagrama normalmente.
- A criptografia foi aplicada corretamente no servidor.
- O servidor retornou o datagrama criptografado para o cliente.
- A resposta foi exibida no cliente, indicando que o processo de criptografia funcionou sem falhas.



```
ubuntu@ubuntu: ~  
at java.base/sun.nio.ch.DatagramChannelImpl.bind(DatagramChannelImpl.  
a:1164)  
at java.base/sun.nio.ch.DatagramSocketAdaptor.bind(DatagramSocketAdap  
.java:107)  
at java.base/java.net.DatagramSocket.createDelegate(DatagramSocket.ja  
1426)  
at java.base/java.net.DatagramSocket.<init>(DatagramSocket.java:330)  
at java.base/java.net.DatagramSocket.<init>(DatagramSocket.java:389)  
at java.base/java.net.DatagramSocket.<init>(DatagramSocket.java:355)  
at ServidorUDP.main(ServidorUDP.java:12)  
ubuntu@ubuntu:~$ sudo lsof -i :6789  
COMMAND  PID  USER  FD  TYPE  DEVICE  SIZE/OFF  NODE  NAME  
java     15978  ubuntu  4u  IPv6  70718      0t0  UDP *:6789  
ubuntu@ubuntu:~$ sudo kill -9 15978  
ubuntu@ubuntu:~$ sudo lsof -i :6789  
[1]+  Killed                  java ServidorUDP  
ubuntu@ubuntu:~$ sudo kill -9 15970  
kill: (15970): No such process  
ubuntu@ubuntu:~$ sudo lsof -i :6789  
ubuntu@ubuntu:~$ java ServidorUDP  
*** Servidor UDP aguardando request  
*** Mensagem recebida: OLA SERVIDOR UDP  
*** Mensagem recebida: OLA SERVIDOR UDP  
200.18.141,127
```



```
Terminal - cefet@LAB-6201B-14: ~
Arquivo  Editar  Ver  Terminal  Abas  Ajuda

cefet@LAB-6201B-14: ~
ClienteUDP.java: comando não encontrado
cefet@LAB-6201B-14:~$ javac ClienteUDP.java
error: file not found: ClienteUDP.java
Usage: javac <options> <source files>
use --help for a list of possible options
cefet@LAB-6201B-14:~$ javac ClienteUDP.java
cefet@LAB-6201B-14:~$ java ClienteUDP
* Resposta criptografada: ROD#VHUYLGRU#XGS
cefet@LAB-6201B-14:~$
cefet@LAB-6201B-14:~$
cefet@LAB-6201B-14:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid lft forever preferred_lft forever
2: eno2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether d0:94:66:dd:d5:e5 brd ff:ff:ff:ff:ff:ff
    altname enp0s31f6
    inet 200.18.140.37/22 brd 200.18.143.255 scope global dynamic noprefixroute
    eno2
```

11. Testes com Regras de Bloqueio no Firewall (IPTables)

Após a validação da criptografia e da comunicação TCP/UDP na máquina real, foram realizadas tentativas de aplicar regras de bloqueio utilizando o IPTables para restringir os protocolos individualmente.

O objetivo era permitir somente UDP em um teste e permitir somente TCP em outro.

11.1. Regras aplicadas

(a) Tentativa de permitir apenas UDP

```
sudo iptables -A INPUT -p udp -s IP_DO_CLIENTE -j ACCEPT
```

```
sudo iptables -A INPUT -p tcp -s IP_DO_CLIENTE -j DROP
```

(b) Tentativa de permitir apenas TCP

```
sudo iptables -A INPUT -p tcp -s IP_DO_CLIENTE -j ACCEPT
```

```
sudo iptables -A INPUT -p udp -s IP_DO_CLIENTE -j DROP
```

11.2. Resultados obtidos

Durante a prática, as regras de bloqueio **não funcionaram como esperado**.

Mesmo com as regras aplicadas, os testes com os clientes TCP e UDP continuavam funcionando normalmente, sem apresentar bloqueio de comunicação.

Possíveis causas observadas

- O sistema operacional pode estar executando outro firewall ativo (como *firewalld*) que sobrepõe ou ignora regras IPTables.
- As regras podem ter sido aplicadas na tabela ou cadeia incorreta.
- Outras regras preexistentes podem ter permitido tráfego antes das regras adicionadas.
- O ambiente pode conter políticas padrão que permitem tráfego local ou em rede interna.

Apesar de diferentes tentativas, o bloqueio não foi efetivo, e ambas as versões — TCP e UDP — continuaram respondendo normalmente.

12. Conclusão Final

A prática foi dividida em duas grandes etapas:

1. **Implementação e testes da criptografia nas comunicações TCP e UDP**
2. **Tentativa de controle de acesso via firewall IPTables**

Funcionamento da criptografia (TCP e UDP)

Esta etapa foi **bem-sucedida**.

Os testes mostraram que:

- A comunicação TCP funcionou corretamente, com envio, retorno e exibição da mensagem criptografada.
- A comunicação UDP também funcionou perfeitamente, recebendo e devolvendo a resposta cifrada.
- A cifra aplicada (César +3) funcionou em todos os cenários.

A implementação do cliente e servidor se mostrou correta tanto em Java quanto no ambiente real.

Tentativas de bloqueio no firewall

Diferente da parte de criptografia, a etapa de bloqueio **não teve sucesso**.

Mesmo após aplicar diversas regras no IPTables, o tráfego TCP e UDP não foi bloqueado como o esperado.

Isso indica que:

- O firewall pode não estar ativo,
- Pode haver outro firewall sobreposto,
- Ou regras prévias podem estar anulando o efeito das novas regras.

Resultado geral

- **Criptografia:** ✓ Funcionou corretamente em TCP e UDP.
- **Firewall (bloqueio):** ✗ Não funcionou, sem efeito no tráfego.

A prática permitiu compreender o funcionamento dos protocolos, a troca de mensagens criptografadas e também destacou que a configuração de firewall depende de diversos fatores do sistema operacional, podendo exigir ajustes adicionais ou o uso de ferramentas complementares.